# 10 Configuring CORS

This chapter covers

- Defining CORS
- Applying CORS configurations

In this chapter, we discuss cross-origin resource sharing (CORS) and how to apply it with Spring Security. First, what is CORS, and why should you care? The need for CORS stems from web applications. By default, browsers don't allow requests made for any domain other than the one from which the site is loaded. For example, if you access the site from example.com, the browser won't let the site make requests to api.example.com. Figure 10.1 illustrates this concept.

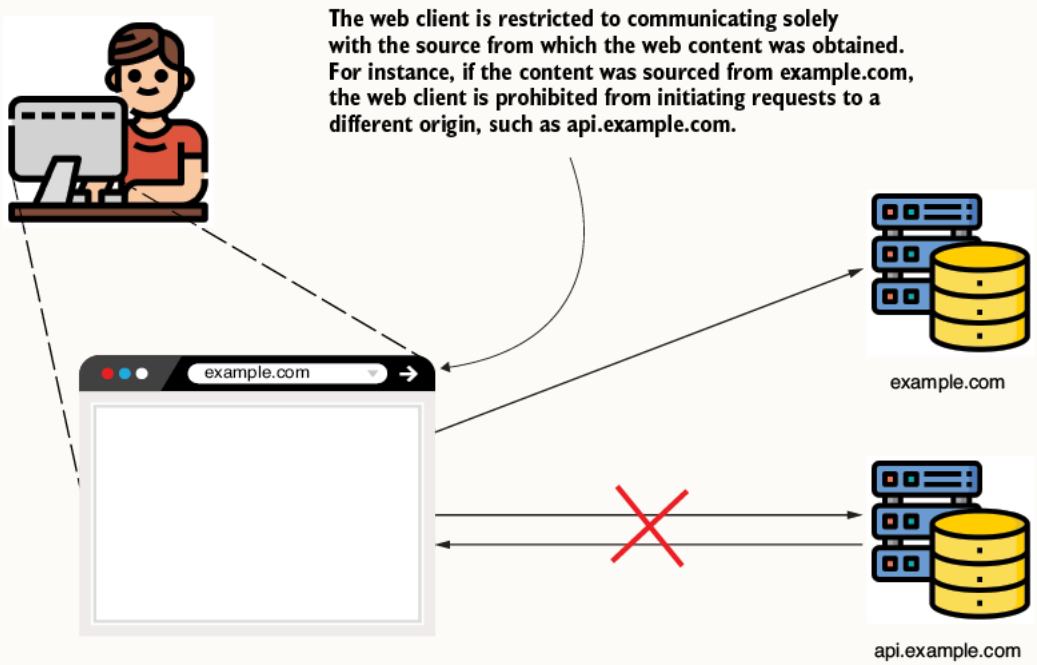



Figure 10.1 Cross-origin resource sharing (CORS). When accessed from example.com, the website cannot make requests to api.example.com because they would be cross-domain requests.

We can briefly say that an app uses the CORS mechanism to relax this strict policy and allow requests made between different origins in some conditions. You need to know this because it's likely you will have to use it for your applications, especially nowadays where the frontend and backend are separate applications. It is common that a frontend application is developed using frameworks such as Angular, ReactJS, or Vue and hosted at a domain such as example.com, but it calls endpoints on the backend hosted at another domain, such as api.example.com.

This chapter provides some examples from which you can learn how to apply CORS policies for your web applications. It also shows how to avoid leaving security breaches in your applications.

## 10.1 How does CORS work?

This section discusses how CORS applies to web applications. If you are the owner of example.com, and for some reason the developers from example.org decide to call your REST endpoints (api.example.com) from their website, they won't be able to. The same situation can happen if a domain loads your application using an `iframe` (see figure 10.2).

**NOTE** An `iframe` is an HTML element used to embed content generated by a web page into another web page (e.g., to integrate the content from example.org inside a page from example.com).

Any situation in which an application makes calls between two different domains is prohibited. Of course, you can encounter situations in which you need to make such calls, and this is when CORS will allow you to specify from which domain your application allows requests and what details can be shared. The CORS mechanism works based on HTTP headers (figure 10.3).

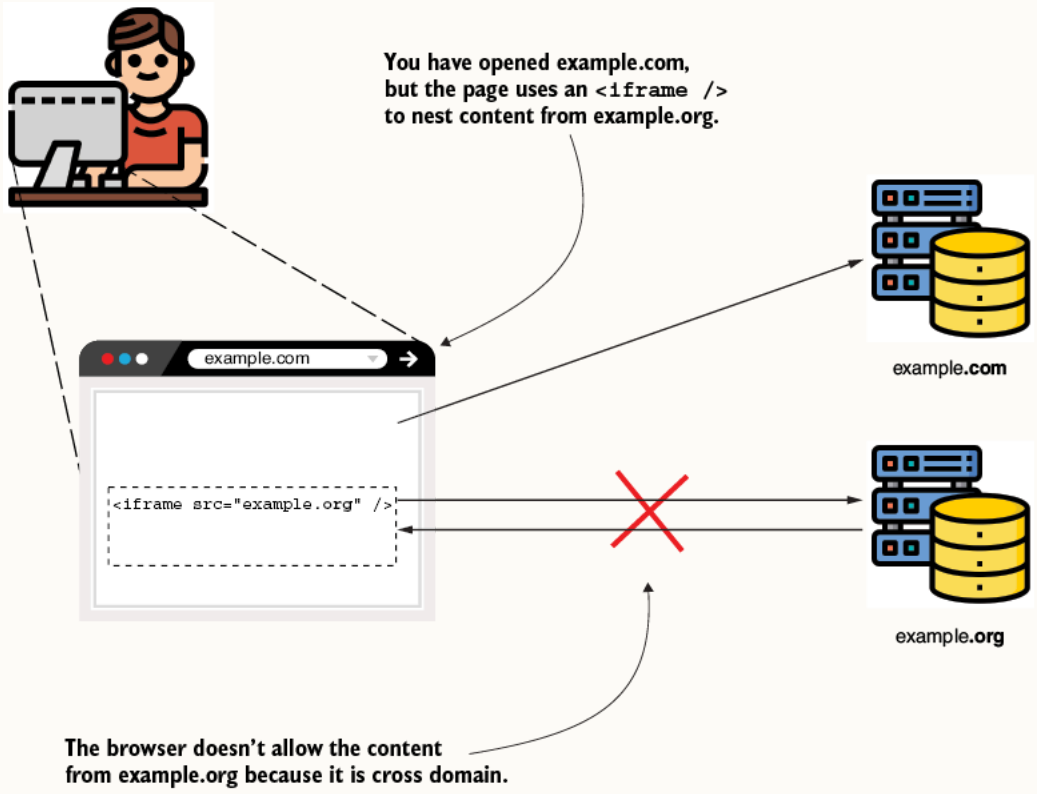



Figure 10.2 Even if the example.org page is loaded in an `iframe` from the example.com domain, the calls from the content loaded in example.org won't load. Furthermore, even if the application makes a request, the browser won't accept the response.

The most important are

- `Access-Control-Allow-Origin` —Specifies the foreign domains (origins) that can access resources on your domain.
- `Access-Control-Allow-Methods` —Lets us refer only to some HTTP methods in situations when we want to allow access to a different domain, but only to specific HTTP methods. For example, you use this if you're going to enable example.com to call some endpoint, but only with HTTP GET.
- `Access-Control-Allow-Headers` —Adds limitations to which headers you can use in a specific request. For example, you don't want the client to be able to send a specific header for a given request.

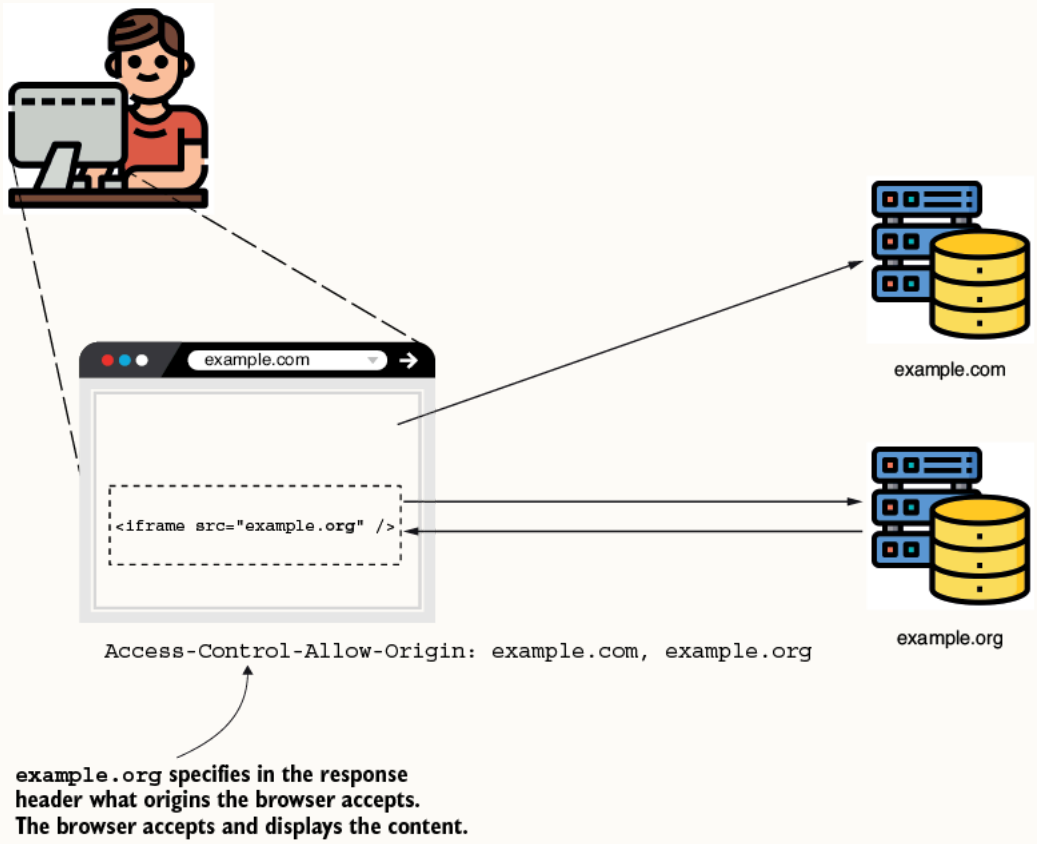



Figure 10.3 Enabling cross-origin requests. The example.org server adds the `Access-Control-Allow-Origin` header to specify the origins of the request for which the browser should accept the response. If the domain from which the call was made is enumerated in the origins, the browser accepts the response.

With Spring Security, by default, none of these headers are added to the response. So let's start at the beginning: What happens when you make a cross-origin call if you don't configure CORS in your application? When the application makes the request, it expects that the response has an `Access-Control-Allow-Origin` header containing the origins accepted by the server. If this doesn't happen, as in the case of default Spring Security behavior, the browser won't accept the response. Let's demonstrate this with a small web application. We create a new project using the dependencies presented by the next code snippet (you can find this example in the project ssia-ch10-ex1):

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

We define a controller class having an action for the main page and a REST endpoint. Because the class is a normal Spring MVC `@Controller` class, we also must add the `@ResponseBody` annotation explicitly to the endpoint. The following listing defines the controller.

Listing 10.1 The definition of the controller class

```
@Controller
public class MainController {

 private Logger logger =                    ①
  Logger.getLogger(MainController.class.getName());

 @GetMapping("/")                           ②
 public String main() {
  return "main.html";
 }

 @PostMapping("/test")
 @ResponseBody
 public String test() {                     ③
  logger.info("Test method called");
  return "HELLO";
 }
}
```

① Uses a logger to observe when the test() method is called

② Defines a main.html page that makes the request to the /test endpoint

③ Defines an endpoint that we call from a different origin to prove how CORS works

Furthermore, we need to define the configuration class where we disable CSRF protection to make the example simpler and allow you to focus on the CORS mechanism only. Additionally, we allow unauthenticated access to all endpoints. The next listing defines this configuration class.

Listing 10.2 The definition of the configuration class

```
@Configuration
public class ProjectConfig {

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http)
  throws Exception {
```

```
    http.csrf(
      c -> c.disable()
    );

    http.authorizeHttpRequests(
      c -> c.anyRequest().permitAll()
    );


    return http.build();
  }
}
```

Of course, we also need to define the main.html file in the resources/templates folder of the project. The main.html file contains the JavaScript code that calls the /test endpoint. To simulate the cross-origin call, we can access the page in a browser using the domain localhost. From the JavaScript code, we make the call using the IP address 127.0.0.1. Even if localhost and 127.0.0.1 refer to the same host, the browser sees these as different strings and considers them different domains. The next listing defines the main.html page.

Listing 10.3 The main.html page

```
<!DOCTYPE HTML>
<html lang=»en»>
  <head>
    <script>
      const http = new XMLHttpRequest();
      const url='http://127.0.0.1:8080/test';    ①
      http.open("POST", url);
      http.send();

      http.onreadystatechange = (e) => {
        document                                 ②
          .getElementById("output")
          .innerHTML = http.responseText;
      }
    </script>
  </head>
  <body>
    <div id="output"></div>
  </body>
</html>
```

① Calls the endpoint using 127.0.0.1 as host to simulate the cross-origin call

② Sets the response body to the output div in the page body

When starting the application and opening the page in a browser with localhost:8080, we can observe that the page doesn't display anything. We expected to see HELLO on the page because this is what the /test endpoint returns. However, when we check the browser console, what we see is an error printed by the JavaScript call. The error looks like this:

```
Access to XMLHttpRequest at 'http://127.0.0.1:8080/test' from origin 'http://localhost:8080' has been blocked by CORS policy: No 'Access-Control-Allo
```

The error message tells us that the response wasn't accepted because the `Access-Control-Allow-Origin` HTTP header doesn't exist. This behavior happens because we didn't configure anything regarding CORS in our Spring Boot application, and by default, it doesn't set any header related to CORS. So the browser's behavior of not displaying the response is correct. I would like you, however, to notice that in the application console, the log proves the method was called. The next code snippet shows what you find in the application console:

```
INFO 25020 --- [nio-8080-exec-2]
➥c.l.s.controllers.MainController :
➥Test method called
```

This aspect is important! I meet many developers who understand CORS as a restriction similar to authorization or CSRF protection. Instead of being a restriction, CORS helps to relax a rigid constraint for cross-domain calls. And even with restrictions applied, in some situations, the endpoint can be called. This behavior doesn't always happen. Sometimes, the browser first makes a call using the HTTP OPTIONS method to test whether the request should be allowed. We call this test request a *preflight* request. If the preflight request fails, the browser won't attempt to honor the original request.

The preflight request and the decision whether to make it are the responsibility of the browser. You don't have to implement this logic. However, it is important to understand it so you won't be surprised to see cross-origin calls to the backend, even if you did not specify any CORS policies for specific domains. This could also happen when you have a client-side app developed with a framework such as Angular or ReactJS. Figure 10.4 presents this request flow. When the browser omits making the preflight request if the HTTP method is GET, POST, or OPTIONS, it only has some basic headers, as described in the official documentation at **https://fetch.spec.whatwg.org/#http-cors-protocol**.

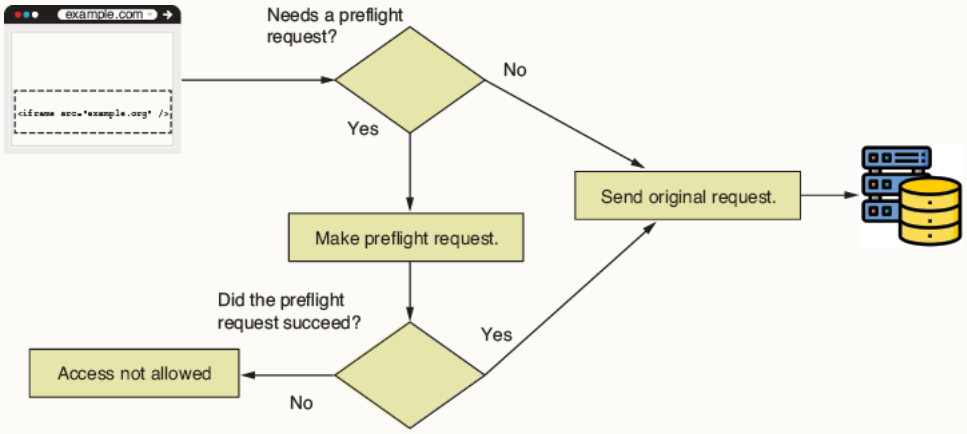



Figure 10.4 For simple requests, the browser sends the original request directly to the server. The browser rejects the response if the server doesn't allow the origin. In some cases, the browser sends a preflight request to test whether the server accepts the origin. If the preflight request succeeds, the browser sends the original request.

In our example, the browser makes the request, but we don't accept the response if the origin is not specified there, as shown in figures 10.1 and 10.2. The CORS mechanism is, in the end, related to the browser and not a way to secure endpoints. The only thing it guarantees is that only origin domains that you allow can make requests from specific pages in the browser.

## 10.2 Applying CORS policies with the @CrossOrigin annotation

This section discusses how to configure CORS to allow requests from different domains using the `@CrossOrigin` annotation. You can place the `@CrossOrigin` annotation directly above the method that defines the endpoint and configure it using the allowed origins and methods. As you learn in this section, the advantage of using the `@CrossOrigin` annotation is that it makes it easy to configure CORS for each endpoint.

We use the application we created in section 10.1 to demonstrate how `@CrossOrigin` works. To make the cross-origin call work in the application, the only thing you need to do is to add the `@CrossOrigin` annotation over the `test()` method in the controller class. The following listing shows how to use the annotation to make the localhost an allowed origin.

Listing 10.4 Making localhost an allowed origin

```
@PostMapping("/test")
@ResponseBody
@CrossOrigin("http://localhost:8080")     ①
public String test() {
  logger.info("Test method called");
  return "HELLO";
}
```

① Allows the localhost origin for cross-origin requests

You can rerun and test the application. This should now display on the page the string returned by the /test endpoint: HELLO.

The value parameter of `@CrossOrigin` receives an array to let you define multiple origins; for example, `@CrossOrigin({"example.com", "example.org"})`. You can also set the allowed headers and methods using the `allowedHeaders` attribute and the `methods` attribute of the annotation. For both origins and headers, you can use the asterisk (*) to represent all headers or

all origins. However, I recommend you exercise caution with this approach. It's always better to filter the origins and headers that you want to allow and never allow any domain to implement code that accesses your applications' resources.

By allowing all origins, you expose the application to cross-site scripting (XSS) requests, which eventually can lead to DDoS attacks. I personally avoid allowing all origins even in test environments. I know that applications sometimes happen to run on wrongly defined infrastructures that use the same data centers for both test and production. It is wiser to treat all layers on which security applies independently, as we discussed in chapter 1, and to avoid assuming that the application doesn't have particular vulnerabilities because the infrastructure doesn't allow it.

The advantage of using `@CrossOrigin` to specify the rules directly where the endpoints are defined is that it creates good transparency of the rules. The disadvantage is that it might become wordy, forcing you to repeat a lot of code. It also imposes the risk that the developer might forget to add the annotation for newly implemented endpoints. In section 10.3, we discuss applying the CORS configuration centralized within the configuration class.

## 10.3 Applying CORS using a CorsConfigurer

Although using the `@CrossOrigin` annotation is easy, as you learned in section 10.2, you might find it more comfortable in a lot of cases to define CORS configuration in one place. In this section, we change the example we worked on in sections 10.1 and 10.2 to apply CORS configuration in the configuration class using a `Customizer`. The next listing shows the changes we need to make in the configuration class to define the origins we want to allow.

Listing 10.5 Defining CORS configurations centralized in the configuration class

```
@Configuration
public class ProjectConfig {

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http)
  throws Exception {

  http.cors(c -> {                          ①
   CorsConfigurationSource source = request -> {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(
       List.of("example.com", "example.org"));
    config.setAllowedMethods(
       List.of("GET", "POST", "PUT", "DELETE"));
    config.setAllowedHeaders(List.of("*"));
    return config;
   };
   c.configurationSource(source);
  });

  http.csrf(
   c -> c.disable()
  );

  http.authorizeHttpRequests(
   c -> c.anyRequest().permitAll()
  );

  return http.build();
 }
}
```

① Calls cors() to define the CORS configuration. Within it, we create a CorsConfiguration object where we set the allowed origins and methods.

The `cors()` method that we call from the `HttpSecurity` object receives as a parameter a `Customizer<CorsConfigurer>` object. For this object, we set a `CorsConfigurationSource`, which returns `CorsConfiguration` for an HTTP request. `CorsConfiguration` is the object that states which are the allowed origins, methods, and headers. If you use this approach, you must at least specify the origins and the methods. If you only specify the origins, your application won't allow the requests. This behavior happens because a `CorsConfiguration` object doesn't define any methods by default.

In this example, to make the explanation straightforward, I provide the implementation for `CorsConfigurationSource` as a lambda expression using the `SecurityFilterChain` bean directly. I strongly recommend separating this code in a different class in your applications. In real-world applications, you could have much longer code, so it could become difficult to read if not separated by the configuration class.

## Summary

- CORS refers to the situation in which a web application hosted on a specific domain tries to access content from another domain.
- By default, the browser doesn't allow cross-origin requests to happen. CORS configuration thus enables you to allow a part of your resources to be called from a different domain in a web application run in the browser.
- You can configure CORS both for an endpoint using the `@CrossOrigin` annotation or centralized in the configuration class using the `cors()` method of the `HttpSecurity` object.

Previous chapter
< 9 Configuring CSRF protection

Next chapter
11 Implementing authorization at the method level >